

Rock Paper Scissors

Complete Source Code

23 files · 757 lines
Spring Boot backend · Vue 3 frontend · JUnit tests

Contents

Build . . . 3

pom.xml . . . 3

Application . . . 3

RockPaperScissorsApplication.java . . . 3

Package: com.example.rps.model . . . 4

Move.java . . . 4

Outcome.java . . . 4

Round.java . . . 5

Scoreboard.java . . . 5

Game.java . . . 6

Package: com.example.rps.player . . . 7

Player.java . . . 7

HumanPlayer.java . . . 7

ComputerPlayer.java . . . 8

Package: com.example.rps.strategy . . . 8

MoveStrategy.java . . . 8

RandomStrategy.java . . . 9

FrequencyStrategy.java . . . 10

Difficulty.java . . . 11

Package: com.example.rps.dto . . . 11

NewGameRequest.java . . . 11

PlayRequest.java . . . 11

GameStateResponse.java . . . 12

Package: com.example.rps.service . . . 13

GameService.java . . . 13

GameNotFoundException.java . . . 14

Package: com.example.rps.controller . . . 14

GameController.java . . . 14

Frontend . . . 15

index.html . . . 15

style.css . . . 18

Tests . . . 19

GameLogicTest.java . . . 19

Build

pom.xml

pom.xml · 43 lines

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/ma
ven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.5</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>rock-paper-scissors</artifactId>
    <version>1.0.0</version>
    <name>rock-paper-scissors</name>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Application

RockPaperScissorsApplication.java

src/main/java/com/example/rps/RockPaperScissorsApplication.java · 11 lines

```java
package com.example.rps;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RockPaperScissorsApplication {
    public static void main(String[] args) {
        SpringApplication.run(RockPaperScissorsApplication.class, args);
    }
}
```

Package: com.example.rps.model

Move.java

src/main/java/com/example/rps/model/Move.java · 43 lines

```java
package com.example.rps.model;

/**
 * The three possible moves. Each move knows which move it beats,
 * so the game rules live in one place instead of a big if/else chain.
 */
public enum Move {
    ROCK("✊"),
    PAPER("✋"),
    SCISSORS("✌");

    private final String emoji;

    Move(String emoji) {
        this.emoji = emoji;
    }

    public String getEmoji() {
        return emoji;
    }

    /** Returns the move that this move defeats. */
    public Move beatsMove() {
        return switch (this) {
            case ROCK -> SCISSORS;
            case PAPER -> ROCK;
            case SCISSORS -> PAPER;
        };
    }

    /** Returns the move that defeats this move. */
    public Move losesTo() {
        return switch (this) {
            case ROCK -> PAPER;
            case PAPER -> SCISSORS;
            case SCISSORS -> ROCK;
        };
    }

    public boolean beats(Move other) {
        return beatsMove() == other;
    }
}
```

Outcome.java

src/main/java/com/example/rps/model/Outcome.java · 11 lines

```java
package com.example.rps.model;

/** Result of a round from the human player's point of view. */
public enum Outcome {
    WIN, LOSE, DRAW;

    public static Outcome decide(Move player, Move computer) {
        if (player == computer) return DRAW;
        return player.beats(computer) ? WIN : LOSE;
    }
}
```

Round.java

src/main/java/com/example/rps/model/Round.java · 5 lines

```java
package com.example.rps.model;

/** Immutable record of one played round. */
public record Round(int number, Move playerMove, Move computerMove, Outcome outcome) {
}
```

Scoreboard.java

src/main/java/com/example/rps/model/Scoreboard.java · 28 lines

```java
package com.example.rps.model;

/**
 * Keeps the running score. Fields are private and can only change
 * through record(), which keeps the score consistent (encapsulation).
 */
public class Scoreboard {
    private int wins;
    private int losses;
    private int draws;

    public void record(Outcome outcome) {
        switch (outcome) {
            case WIN -> wins++;
            case LOSE -> losses++;
            case DRAW -> draws++;
        }
    }

    public void reset() {
        wins = losses = draws = 0;
    }

    public int getWins() { return wins; }
    public int getLosses() { return losses; }
    public int getDraws() { return draws; }
    public int getTotal() { return wins + losses + draws; }
}
```

Game.java

src/main/java/com/example/rps/model/Game.java · 52 lines

```java
package com.example.rps.model;

import com.example.rps.player.ComputerPlayer;
import com.example.rps.player.HumanPlayer;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * A game session between a human and the computer.
 * It is composed of players, a scoreboard and a round history (composition).
 */
public class Game {
    private final HumanPlayer human;
    private final ComputerPlayer computer;
    private final Scoreboard scoreboard = new Scoreboard();
    private final List<Round> history = new ArrayList<>();

    public Game(HumanPlayer human, ComputerPlayer computer) {
        this.human = human;
        this.computer = computer;
    }

    public Round playRound(Move humanMove) {
        human.setNextMove(humanMove);

        Move playerMove = human.chooseMove();
        Move computerMove = computer.chooseMove();
        Outcome outcome = Outcome.decide(playerMove, computerMove);

        Round round = new Round(history.size() + 1, playerMove, computerMove, outcome);
        history.add(round);
        scoreboard.record(outcome);
        computer.observeOpponent(playerMove);
        return round;
    }

    public void reset() {
        history.clear();
        scoreboard.reset();
        computer.forget();
    }

    public HumanPlayer getHuman() { return human; }
    public ComputerPlayer getComputer() { return computer; }
    public Scoreboard getScoreboard() { return scoreboard; }

    public List<Round> getHistory() {
        return Collections.unmodifiableList(history);
    }
}
```

Package: com.example.rps.player

Player.java

src/main/java/com/example/rps/player/Player.java · 21 lines

```java
package com.example.rps.player;

import com.example.rps.model.Move;

/**
 * Abstract base class for anyone who can play (abstraction).
 * Subclasses decide HOW a move is chosen.
 */
public abstract class Player {
    private final String name;

    protected Player(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract Move chooseMove();
}
```

HumanPlayer.java

src/main/java/com/example/rps/player/HumanPlayer.java · 24 lines

```java
package com.example.rps.player;

import com.example.rps.model.Move;

/** A human player: the move comes from the UI. */
public class HumanPlayer extends Player {
    private Move nextMove;

    public HumanPlayer(String name) {
        super(name);
    }

    public void setNextMove(Move move) {
        this.nextMove = move;
    }

    @Override
    public Move chooseMove() {
        if (nextMove == null) {
            throw new IllegalStateException("Human player has not selected a move");
        }
        return nextMove;
    }
}
```

ComputerPlayer.java

src/main/java/com/example/rps/player/ComputerPlayer.java · 34 lines

```java
package com.example.rps.player;

import com.example.rps.model.Move;
import com.example.rps.strategy.MoveStrategy;

/**
 * A computer player that delegates its decision to a pluggable
 * MoveStrategy (Strategy design pattern + polymorphism).
 */
public class ComputerPlayer extends Player {
    private final MoveStrategy strategy;

    public ComputerPlayer(String name, MoveStrategy strategy) {
        super(name);
        this.strategy = strategy;
    }

    @Override
    public Move chooseMove() {
        return strategy.nextMove();
    }

    public void observeOpponent(Move opponentMove) {
        strategy.observe(opponentMove);
    }

    public void forget() {
        strategy.reset();
    }

    public String getStrategyName() {
        return strategy.getName();
    }
}
```

Package: com.example.rps.strategy

MoveStrategy.java

src/main/java/com/example/rps/strategy/MoveStrategy.java · 17 lines

```java
package com.example.rps.strategy;

import com.example.rps.model.Move;

/** How a computer player picks its move. */
public interface MoveStrategy {
    Move nextMove();

    String getName();

    /** Lets "learning" strategies see what the opponent played. Optional. */
    default void observe(Move opponentMove) {
    }

    default void reset() {
    }
}
```

RandomStrategy.java

src/main/java/com/example/rps/strategy/RandomStrategy.java · 29 lines

```java
package com.example.rps.strategy;

import com.example.rps.model.Move;

import java.util.Random;

/** Picks a completely random move — the classic fair opponent. */
public class RandomStrategy implements MoveStrategy {
    private final Random random;

    public RandomStrategy() {
        this(new Random());
    }

    public RandomStrategy(Random random) {
        this.random = random;
    }

    @Override
    public Move nextMove() {
        Move[] moves = Move.values();
        return moves[random.nextInt(moves.length)];
    }

    @Override
    public String getName() {
        return "Easy (random)";
    }
}
```

FrequencyStrategy.java

src/main/java/com/example/rps/strategy/FrequencyStrategy.java · 51 lines

```java
package com.example.rps.strategy;

import com.example.rps.model.Move;

import java.util.EnumMap;
import java.util.Map;

/**
 * "Smart" opponent: remembers which move you play most often and
 * plays the move that beats it. Falls back to random with no data.
 */
public class FrequencyStrategy implements MoveStrategy {
    private final Map<Move, Integer> counts = new EnumMap<>(Move.class);
    private final RandomStrategy fallback;

    public FrequencyStrategy() {
        this(new RandomStrategy());
    }

    public FrequencyStrategy(RandomStrategy fallback) {
        this.fallback = fallback;
    }

    @Override
    public Move nextMove() {
        Move favourite = null;
        int best = 0;
        for (Map.Entry<Move, Integer> e : counts.entrySet()) {
            if (e.getValue() > best) {
                best = e.getValue();
                favourite = e.getKey();
            }
        }
        return favourite == null ? fallback.nextMove() : favourite.losesTo();
    }

    @Override
    public void observe(Move opponentMove) {
        counts.merge(opponentMove, 1, Integer::sum);
    }

    @Override
    public void reset() {
        counts.clear();
    }

    @Override
    public String getName() {
        return "Hard (learns your habits)";
    }
}
```

Difficulty.java

src/main/java/com/example/rps/strategy/Difficulty.java · 19 lines

```java
package com.example.rps.strategy;

import java.util.function.Supplier;

/** Maps a difficulty level to a strategy factory. */
public enum Difficulty {
    EASY(RandomStrategy::new),
    HARD(FrequencyStrategy::new);

    private final Supplier<MoveStrategy> factory;

    Difficulty(Supplier<MoveStrategy> factory) {
        this.factory = factory;
    }

    public MoveStrategy createStrategy() {
        return factory.get();
    }
}
```

Package: com.example.rps.dto

NewGameRequest.java

src/main/java/com/example/rps/dto/NewGameRequest.java · 6 lines

```java
package com.example.rps.dto;

import com.example.rps.strategy.Difficulty;

public record NewGameRequest(String playerName, Difficulty difficulty) {
}
```

PlayRequest.java

src/main/java/com/example/rps/dto/PlayRequest.java · 6 lines

```java
package com.example.rps.dto;

import com.example.rps.model.Move;

public record PlayRequest(Move move) {
}
```

GameStateResponse.java

src/main/java/com/example/rps/dto/GameStateResponse.java · 32 lines

```java
package com.example.rps.dto;

import com.example.rps.model.Game;
import com.example.rps.model.Round;
import com.example.rps.model.Scoreboard;

import java.util.List;

/** What the UI receives after every action. */
public record GameStateResponse(
        String gameId,
        String playerName,
        String opponent,
        Round lastRound,
        int wins,
        int losses,
        int draws,
        List<Round> history) {

    public static GameStateResponse from(String id, Game game) {
        Scoreboard s = game.getScoreboard();
        List<Round> history = game.getHistory();
        Round last = history.isEmpty() ? null : history.get(history.size() - 1);
        return new GameStateResponse(
                id,
                game.getHuman().getName(),
                game.getComputer().getStrategyName(),
                last,
                s.getWins(), s.getLosses(), s.getDraws(),
                history);
    }
}
```

Package: com.example.rps.service

GameService.java

src/main/java/com/example/rps/service/GameService.java · 54 lines

```java
package com.example.rps.service;

import com.example.rps.model.Game;
import com.example.rps.model.Move;
import com.example.rps.player.ComputerPlayer;
import com.example.rps.player.HumanPlayer;
import com.example.rps.strategy.Difficulty;
import org.springframework.stereotype.Service;

import java.util.Map;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;

/** Business layer: creates and runs games. Games are kept in memory. */
@Service
public class GameService {
    private final Map<String, Game> games = new ConcurrentHashMap<>();

    public String createGame(String playerName, Difficulty difficulty) {
        String name = (playerName == null || playerName.isBlank()) ? "Player" : playerName.tri
m();
        Difficulty level = difficulty == null ? Difficulty.EASY : difficulty;

        Game game = new Game(
                new HumanPlayer(name),
                new ComputerPlayer("Computer", level.createStrategy()));

        String id = UUID.randomUUID().toString();
        games.put(id, game);
        return id;
    }

    public Game getGame(String id) {
        Game game = games.get(id);
        if (game == null) throw new GameNotFoundException(id);
        return game;
    }

    public Game play(String id, Move move) {
        if (move == null) throw new IllegalArgumentException("Move is required");
        Game game = getGame(id);
        synchronized (game) {
            game.playRound(move);
        }
        return game;
    }

    public Game reset(String id) {
        Game game = getGame(id);
        synchronized (game) {
            game.reset();
        }
        return game;
    }
}
```

GameNotFoundException.java

src/main/java/com/example/rps/service/GameNotFoundException.java · 11 lines

```java
package com.example.rps.service;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class GameNotFoundException extends RuntimeException {
    public GameNotFoundException(String id) {
        super("Game not found: " + id);
    }
}
```

Package: com.example.rps.controller

GameController.java

src/main/java/com/example/rps/controller/GameController.java · 47 lines

```java
package com.example.rps.controller;

import com.example.rps.dto.GameStateResponse;
import com.example.rps.dto.NewGameRequest;
import com.example.rps.dto.PlayRequest;
import com.example.rps.service.GameService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

/** REST API used by the Vue front-end. */
@RestController
@RequestMapping("/api/games")
public class GameController {
    private final GameService service;

    public GameController(GameService service) {
        this.service = service;
    }

    @PostMapping
    public GameStateResponse newGame(@RequestBody NewGameRequest req) {
        String id = service.createGame(req.playerName(), req.difficulty());
        return GameStateResponse.from(id, service.getGame(id));
    }

    @GetMapping("/{id}")
    public GameStateResponse get(@PathVariable String id) {
        return GameStateResponse.from(id, service.getGame(id));
    }

    @PostMapping("/{id}/play")
    public GameStateResponse play(@PathVariable String id, @RequestBody PlayRequest req) {
        return GameStateResponse.from(id, service.play(id, req.move()));
    }

    @PostMapping("/{id}/reset")
    public GameStateResponse reset(@PathVariable String id) {
        return GameStateResponse.from(id, service.reset(id));
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String, String>> badRequest(IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
    }
}
```

Frontend

index.html

src/main/resources/static/index.html · 125 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Rock Paper Scissors</title>
  <script src="https://unpkg.com/vue@3.4.38/dist/vue.global.prod.js"></script>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<div id="app" class="container">
  <h1>✊ ✋ ✌️ Rock Paper Scissors</h1>

  <!-- Setup screen -->
  <section v-if="!game" class="card">
    <h2>New game</h2>
    <label>Your name
      <input v-model="playerName" placeholder="Player" @keyup.enter="startGame">
    </label>
    <label>Difficulty
      <select v-model="difficulty">
        <option value="EASY">Easy — random computer</option>
        <option value="HARD">Hard — computer learns your habits</option>
      </select>
    </label>
    <button class="primary" @click="startGame">Start</button>
  </section>

  <!-- Game screen -->
  <section v-else class="card">
    <p class="vs"><b>{{ game.playerName }}</b> vs <b>Computer</b> <small>({{ game.opponent }})
</small></p>

    <div class="score">
      <div class="win"><span>{{ game.wins }}</span>Wins</div>
      <div class="draw"><span>{{ game.draws }}</span>Draws</div>
      <div class="lose"><span>{{ game.losses }}</span>Losses</div>
    </div>

    <div class="moves">
      <button v-for="m in moves" :key="m.value" class="move" :disabled="busy" @click="play(m.v
alue)">
        <span class="emoji">{{ m.emoji }}</span>{{ m.label }}
      </button>
    </div>
```

```
44
45      <div v-if="game.lastRound" class="result" :class="game.lastRound.outcome.toLowerCase()">
46        <div class="faceoff">
47          <span>{{ emoji(game.lastRound.playerMove) }}</span>
48          <small>vs</small>
49          <span>{{ emoji(game.lastRound.computerMove) }}</span>
50        </div>
51        <h2>{{ message(game.lastRound.outcome) }}</h2>
52      </div>
53
54      <div class="actions">
55        <button @click="reset">Reset score</button>
56        <button @click="game = null">Change settings</button>
57      </div>
58
59      <table v-if="game.history.length">
60        <thead><tr><th>#</th><th>You</th><th>Computer</th><th>Result</th></tr></thead>
61        <tbody>
62          <tr v-for="r in [...game.history].reverse()" :key="r.number">
63            <td>{{ r.number }}</td>
64            <td>{{ emoji(r.playerMove) }}</td>
65            <td>{{ emoji(r.computerMove) }}</td>
66            <td :class="r.outcome.toLowerCase()">{{ r.outcome }}</td>
67          </tr>
68        </tbody>
69      </table>
70    </section>
71
72    <p v-if="error" class="error">{{ error }}</p>
73  </div>
74
75  <script>
76  const { createApp } = Vue;
77
78  const MOVES = [
79    { value: 'ROCK', label: 'Rock', emoji: '✊' },
80    { value: 'PAPER', label: 'Paper', emoji: '✋' },
81    { value: 'SCISSORS', label: 'Scissors', emoji: '✌️' }
82  ];
83
84  createApp({
85    data() {
86      return { playerName: '', difficulty: 'EASY', game: null, busy: false, error: '', moves: MO
 ↪ VES };
87    },
```

```
  methods: {
    async api(path, body) {
      this.error = '';
      this.busy = true;
      try {
        const res = await fetch('/api/games' + path, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(body || {})
        });
        if (!res.ok) throw new Error('Server error ' + res.status);
        this.game = await res.json();
      } catch (e) {
        this.error = e.message;
      } finally {
        this.busy = false;
      }
    },
    startGame() {
      return this.api('', { playerName: this.playerName, difficulty: this.difficulty });
    },
    play(move) {
      return this.api(`/${this.game.gameId}/play`, { move });
    },
    reset() {
      return this.api(`/${this.game.gameId}/reset`);
    },
    emoji(move) {
      return MOVES.find(m => m.value === move).emoji;
    },
    message(outcome) {
      return { WIN: 'You win! 🎉', LOSE: 'You lose 😢', DRAW: "It's a draw 🤝" }[outcome];
    }
  }
}).mount('#app');
</script>
</body>
</html>
```

style.css

src/main/resources/static/style.css · 32 lines

```
:root {
  --bg: #f4f6fb; --card: #fff; --text: #1f2430; --muted: #6b7280;
  --accent: #4f46e5; --win: #16a34a; --lose: #dc2626; --draw: #d97706;
}
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: var(--bg); color: var(--text
); }
.container { max-width: 640px; margin: 0 auto; padding: 24px 16px; }
h1 { text-align: center; font-size: 1.6rem; }
.card { background: var(--card); border-radius: 14px; padding: 24px; box-shadow: 0 4px 18px rg
ba(0,0,0,.07); }
label { display: block; margin-bottom: 14px; font-weight: 600; }
input, select { display: block; width: 100%; margin-top: 6px; padding: 10px; font-size: 1rem;
  border: 1px solid #d1d5db; border-radius: 8px; }
button { padding: 10px 16px; font-size: 1rem; border: 1px solid #d1d5db; border-radius: 8px;
  background: #fff; cursor: pointer; }
button.primary { background: var(--accent); color: #fff; border: none; width: 100%; }
button:disabled { opacity: .5; cursor: wait; }
.vs { text-align: center; color: var(--muted); }
.score { display: flex; gap: 10px; margin: 16px 0; }
.score div { flex: 1; text-align: center; padding: 10px; border-radius: 10px; background: var(
--bg); font-size: .85rem; }
.score span { display: block; font-size: 1.8rem; font-weight: 700; }
.win { color: var(--win); } .lose { color: var(--lose); } .draw { color: var(--draw); }
.moves { display: flex; gap: 10px; }
.move { flex: 1; display: flex; flex-direction: column; align-items: center; padding: 16px 8px
; transition: transform .1s; }
.move:hover:not(:disabled) { transform: translateY(-3px); border-color: var(--accent); }
.emoji { font-size: 2.4rem; }
.result { text-align: center; margin: 20px 0; }
.faceoff { font-size: 3rem; display: flex; justify-content: center; align-items: center; gap:
16px; }
.faceoff small { font-size: 1rem; color: var(--muted); }
.actions { display: flex; gap: 10px; justify-content: center; margin: 16px 0; }
table { width: 100%; border-collapse: collapse; margin-top: 10px; }
th, td { padding: 6px; text-align: center; border-bottom: 1px solid #eee; }
.error { color: var(--lose); text-align: center; }
```

Tests

GameLogicTest.java

src/test/java/com/example/rps/GameLogicTest.java · 56 lines

```java
package com.example.rps;

import com.example.rps.model.Game;
import com.example.rps.model.Move;
import com.example.rps.model.Outcome;
import com.example.rps.model.Round;
import com.example.rps.player.ComputerPlayer;
import com.example.rps.player.HumanPlayer;
import com.example.rps.strategy.FrequencyStrategy;
import com.example.rps.strategy.MoveStrategy;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class GameLogicTest {

    @Test
    void rulesAreCorrect() {
        assertEquals(Outcome.WIN, Outcome.decide(Move.ROCK, Move.SCISSORS));
        assertEquals(Outcome.WIN, Outcome.decide(Move.PAPER, Move.ROCK));
        assertEquals(Outcome.WIN, Outcome.decide(Move.SCISSORS, Move.PAPER));
        assertEquals(Outcome.LOSE, Outcome.decide(Move.ROCK, Move.PAPER));
        assertEquals(Outcome.DRAW, Outcome.decide(Move.PAPER, Move.PAPER));
    }

    @Test
    void gameTracksScoreAndHistory() {
        MoveStrategy alwaysRock = new MoveStrategy() {
            public Move nextMove() { return Move.ROCK; }
            public String getName() { return "Rock"; }
        };
        Game game = new Game(new HumanPlayer("Ann"), new ComputerPlayer("CPU", alwaysRock));

        Round r = game.playRound(Move.PAPER);
        game.playRound(Move.SCISSORS);
        game.playRound(Move.ROCK);

        assertEquals(Outcome.WIN, r.outcome());
        assertEquals(1, game.getScoreboard().getWins());
        assertEquals(1, game.getScoreboard().getLosses());
        assertEquals(1, game.getScoreboard().getDraws());
        assertEquals(3, game.getHistory().size());

        game.reset();
        assertEquals(0, game.getScoreboard().getTotal());
    }

    @Test
    void frequencyStrategyCountersFavouriteMove() {
        FrequencyStrategy s = new FrequencyStrategy();
        s.observe(Move.ROCK);
        s.observe(Move.ROCK);
        s.observe(Move.PAPER);
        assertEquals(Move.PAPER, s.nextMove());
    }
}
```